```
    if done:
        ep_lengths.append(info['x_pos'])
        state1 = reset_env()
        last_x_pos = env.env.env._x_position
        episode_length = 0
    else:
        state1 = state2
    if len(replay.memory) < params['batch_size']:
        continue
    forward_pred_err, inverse_pred_err, q_loss =
minibatch_train(use_extrinsic=False)                          8
    loss = loss_fn(q_loss, forward_pred_err, inverse_pred_err)   9
    loss_list = (q_loss.mean(), forward_pred_err.flatten().mean(),\
                 inverse_pred_err.flatten().mean())
    losses.append(loss_list)
    loss.backward()
    opt.step()
```

- ***1*** We need to keep track of the last x position in order to reset if there's no forward progress.
- ***2*** Runs the DQN forward to get action-value predictions
- ***3*** After the first 1,000 epochs, switches to the epsilon-greedy policy
- ***4*** Repeats whatever action the policy says 6 times, to speed up learning
- ***5*** Converts the deque object into a tensor
- ***6*** Adds the single experience to the replay buffer
- ***7*** If Mario is not making sufficient forward progress, restarts the game and tries again
- ***8*** Gets the errors for one mini-batch of data from the replay buffer
- ***9*** Computes the overall loss

While it's a bit lengthy, this training loop is pretty simple. All we do is prepare a state, input to the DQN, get action values (Q values), input to the policy, get an action to take, and then call the `env.step(action)` method to perform the action. We then get the next state and some other metadata. We add this full experience as a tuple, (S_t,a_t,r_t,S_{t+1}), to the experience replay memory. Most of the action is happening in the mini-batch training function we already covered.

That is the main code you need to build an end-to-end DQN and ICM to train on Super Mario Bros. Let's test it out by training for 5,000 epochs, which takes about 30 minutes or so running on a MacBook Air (with no